

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 2**



Android Layout With Compose

Oleh:

Muhammad Guntur Ricky Adhitya NIM. 2410817310003

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026**

LEMBAR PENGESAHAN

LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE

MODUL 2

Laporan Praktikum Pemrograman Mobile Modul 2: Android Layout With Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Guntur Ricky Adhitya

NIM : 2410817310003

Menyetujui,
Asisten Praktikum

Menyetujui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom., M.Kom.
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI.....	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL.....	5
SOAL 1.....	6
A. Source Code.....	8
B. Output Program.....	22
C. Pembahasan.....	26
D. Essay.....	28
DAFTAR PUSTAKA.....	31
LINK GITHUB.....	32

DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi.....	6
Gambar 2. Tampilan Pilihan Persentase Tip.....	7
Gambar 3. Tampilan Aplikasi Setelah Dijalankan.....	7
Gambar 4. Screenshot Output Soal 1 XML Modul 2 (1).....	22
Gambar 5. Screenshot Output Soal 1 XML Modul 2 (2).....	23
Gambar 6. Screenshot Output Soal 1 XML Modul 2 (3).....	23
Gambar 7. Screenshot Output Soal 1 XML Modul 2 (4).....	24
Gambar 8. Screenshot Output Soal 1 XML Modul 2 (5).....	24
Gambar 9. Screenshot Output Soal 1 Compose Modul 2 (1).....	25
Gambar 10. Screenshot Output Soal 1 Compose Modul 2 (2).....	25
Gambar 11. Screenshot Output Soal 1 Compose Modul 2 (3).....	26
Gambar 12. Screenshot Output Soal 1 Compose Modul 2 (4).....	26

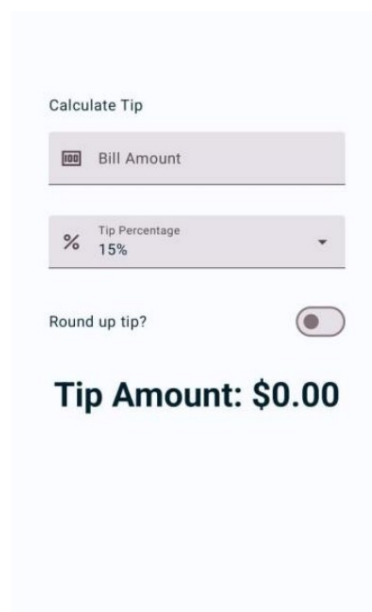
DAFTAR TABEL

Tabel 1 Source Code Layout XML Soal 1 Modul 2.....	8
Tabel 2. Source Code MainActivity XML Soal 1 Modul 2.....	11
Tabel 3. Source Code TipCalculatorController XML Soal 1 Modul 2.....	13
Tabel 4. Source Code TipCalculatorModel XML Soal 1 Modul 2.....	14
Tabel 5. Source Code MainActivity Compose Soal 1 Modul 2.....	15
Tabel 6. Source Code TipPercentageDropdown Soal 1 Modul 2.....	18
Tabel 7. Source Code TipCalculatorController Soal 1 Modul 2.....	20
Tabel 8. Source Code TipCalculatorModel Soal 1 Modul 2.....	21
Tabel 9. Source Code Java Data Model Modul 2.....	29
Tabel 10. Source Code Kotlin NotificationService Modul 2.....	29
Tabel 11. Source Code Java Main Modul 2.....	30

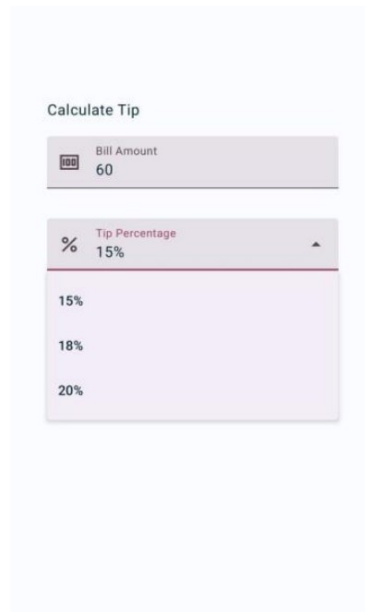
SOAL 1

Soal Praktikum:

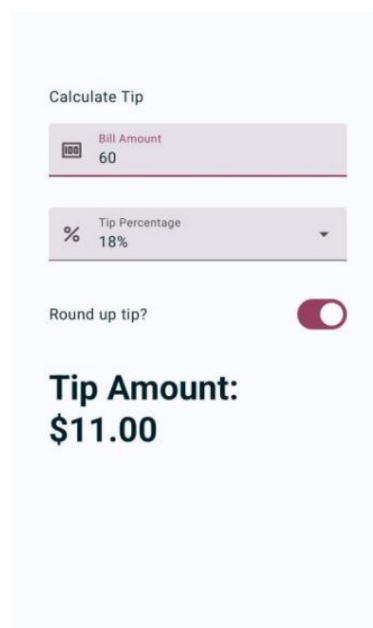
1. Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:
 - a. Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
 - b. Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.
 - c. Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
 - d. Tampilan hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.
2. Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.



Gambar 1. Tampilan Awal Aplikasi



Gambar 2. Tampilan Pilihan Persentase Tip



Gambar 3. Tampilan Aplikasi Setelah Dijalankan

Soal Essay:

1. Apa saja fitur yang menjadi daya jual pada Kotlin? (contoh: null safety, chain, dll)**
2. berikan 1 test case terkait penggunaan fitur unggulan yang ada pada kotlin (contoh: penggunaan null safety)

A. Source Code

Tabel 1 Source Code Layout XML Soal 1 Modul 2

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/androi d"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:id="@+id/main"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	android:padding="32dp"
9	tools:context=".MainActivity">
10	
11	<TextView
12	android:id="@+id/tv_title"
13	android:layout_width="wrap_content"
14	android:layout_height="wrap_content"
15	android:text="@string/calculate_tip"
16	android:textSize="24sp"
17	android:textStyle="bold"
18	app:layout_constraintTop_toTopOf="parent"
19	app:layout_constraintStart_toStartOf="parent"
20	app:layout_constraintBottom_toTopOf="@id/til_bil l_amount"
21	app:layout_constraintVertical_chainStyle="packed "
22	app:layout_constraintVertical_bias="0.5" />
23	
24	<com.google.android.material.textfield.TextInputLayo ut
25	android:id="@+id/til_bill_amount"
26	style="@style/Widget.MaterialComponents.TextInput tLayout.OutlinedBox"

```

27         android:layout_width="0dp"
28         android:layout_height="wrap_content"
29         android:layout_marginTop="16dp"
30         android:hint="@string/bill_amount"
31         app:startIconDrawable="@drawable/ic_money"
32         app:layout_constraintTop_toBottomOf="@id/tv_title"
33         app:layout_constraintStart_toStartOf="parent"
34         app:layout_constraintEnd_toEndOf="parent"
35         app:layout_constraintBottom_toTopOf="@id/til_tip_percentage">
36
37         <com.google.android.material.textfield.TextInput
38         EditText
39
40             android:id="@+id/et_bill_amount"
41             android:layout_width="match_parent"
42             android:layout_height="wrap_content"
43             android:inputType="numberDecimal"
44             android:singleLine="true" />
45     </com.google.android.material.textfield.TextInputLayout>
46
47     <com.google.android.material.textfield.TextInputLayout
48     ut
49
50         android:id="@+id/til_tip_percentage"
51         style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox.ExposedDropDownMenu"
52         android:layout_width="0dp"
53         android:layout_height="wrap_content"
54         android:layout_marginTop="16dp"
55         android:hint="@string/tip_percentage"
56         app:startIconDrawable="@drawable/ic_percent"
57         app:layout_constraintTop_toBottomOf="@id/til_bill_amount"
58         app:layout_constraintStart_toStartOf="parent"
59         app:layout_constraintEnd_toEndOf="parent"

```

```

56         app:layout_constraintBottom_toTopOf="@id/tv_round_up">
57
58         <AutoCompleteTextView
59             android:id="@+id/actv_tip_percentage"
60             android:layout_width="match_parent"
61             android:layout_height="wrap_content"
62             android:inputType="none"
63             android:text="15%"
64             tools:ignore="LabelFor" />
65     </com.google.android.material.textfield.TextInputLayout>
66
67     <TextView
68         android:id="@+id/tv_round_up"
69         android:layout_width="match_parent"
70         android:layout_height="wrap_content"
71         android:layout_marginTop="16dp"
72         android:text="@string/round_up_tip"
73         android:textSize="16sp"
74         app:layout_constraintTop_toBottomOf="@id/til_tip_percentage"
75         app:layout_constraintStart_toStartOf="parent"
76         app:layout_constraintBottom_toTopOf="@id/tv_tip_amount"
77         app:layout_constraintEnd_toStartOf="@id/switch_round_up" />
78
79     <com.google.android.material.switchmaterial.SwitchMaterial
80         android:id="@+id/switch_round_up"
81         android:layout_width="wrap_content"
82         android:layout_height="wrap_content"
83         android:checked="true"
84         app:layout_constraintTop_toTopOf="@id/tv_round_up"

```

85	app:layout_constraintBottom_toBottomOf="@id/tv_r
	ound_up"
86	app:layout_constraintEnd_toEndOf="parent" />
87	
88	<TextView
89	android:id="@+id/tv_tip_amount"
90	android:layout_width="wrap_content"
91	android:layout_height="wrap_content"
92	android:layout_marginTop="16dp"
93	android:text="@string/tip_amount_default"
94	android:textSize="32sp"
95	android:textStyle="bold"
96	app:layout_constraintTop_toBottomOf="@id/tv_roun
	d_up"
97	app:layout_constraintStart_toStartOf="parent"
98	app:layout_constraintBottom_toBottomOf="parent"
	/>
99	
100	</androidx.constraintlayout.widget.ConstraintLayout>

Tabel 2. Source Code MainActivity XML Soal 1 Modul 2

1	package com.mobil.modul2xml
2	
3	import android.os.Bundle
4	import android.widget.ArrayAdapter
5	import androidx.appcompat.app.AppCompatActivity
6	import androidx.core.widget.doAfterTextChanged
7	import
	com.mobil.modul2xml.databinding.ActivityMainBinding
8	
9	class MainActivity : AppCompatActivity() {
10	
11	private lateinit var binding: ActivityMainBinding
12	private val controller = TipCalculatorController()

```

13
14     override fun onCreate(savedInstanceState: Bundle?) {
15         super.onCreate(savedInstanceState)
16                                     binding =
ActivityMainBinding.inflate(layoutInflater)
17         setContentView(binding.root)
18
19         setupDropdown()
20         setupListeners()
21         updateTipDisplay()
22     }
23
24     private fun setupDropdown() {
25         val adapter = ArrayAdapter(
26             this,
27             android.R.layout.simple_dropdown_item_1line,
28             controller.tipOptions.map { "$it%" }
29         )
30         binding.actvTipPercentage.setAdapter(adapter)
31
32         binding.actvTipPercentage.setText("$
{controller.selectedPercentage}%", false)
33     }
34
35     private fun setupListeners() {
36         binding.etBillAmount.doAfterTextChanged { text
->
37             controller.updateBillAmount(text.toString())
38             updateTipDisplay()
39         }
40
41         binding.actvTipPercentage.setOnItemClickListener
{ _, _, position, _ ->
42             controller.updatePercentage(controller.tipOp
tions[position])

```

43	updateTipDisplay()
44	}
45	
46	binding.switchRoundUp.setOnCheckedChangeListener
	{ _, isChecked ->
47	controller.toggleRoundUp(isChecked)
48	updateTipDisplay()
49	}
50	}
51	
52	private fun updateTipDisplay() {
53	binding.tvTipAmount.text =
	getString(R.string.tip_amount_result,
	controller.getTipAmount())
54	}
55	}

Tabel 3. Source Code TipCalculatorController XML Soal 1 Modul 2

1	package com.mobil.modul2xml
2	
3	import java.math.BigDecimal
4	
5	class TipCalculatorController {
6	
7	val tipOptions: List<String> = listOf("15", "18",
	"20")
8	
9	var billAmountInput: String = ""
10	var selectedPercentage: String = "15"
11	var isRoundUp: Boolean = true
12	
13	fun getTipAmount(): String {
14	val amount =
	billAmountInput.toBigDecimalOrNull() ?: BigDecimal.ZERO
15	val percentage =

	<code>selectedPercentage.toBigDecimal().divide(BigDecimal(100))</code>
16	<code>val tip = TipCalculatorModel.calculateTip(amount, percentage, isRoundUp)</code>
17	<code>return tip.toString()</code>
18	<code>}</code>
19	
20	<code>fun updateBillAmount(input: String) {</code>
21	<code>billAmountInput = input</code>
22	<code>}</code>
23	
24	<code>fun updatePercentage(newPercent: String) {</code>
25	<code>selectedPercentage = newPercent</code>
26	<code>}</code>
27	
28	<code>fun toggleRoundUp(enabled: Boolean) {</code>
29	<code>isRoundUp = enabled</code>
30	<code>}</code>
31	<code>}</code>

Tabel 4. Source Code TipCalculatorModel XML Soal 1 Modul 2

1	<code>package com.mobil.modul2xml</code>
2	
3	<code>import java.math.BigDecimal</code>
4	<code>import java.math.RoundingMode</code>
5	
6	<code>object TipCalculatorModel {</code>
7	<code>fun calculateTip(</code>
8	<code>amount: BigDecimal,</code>
9	<code>percentage: BigDecimal,</code>
10	<code>isRoundUp: Boolean</code>
11	<code>): BigDecimal {</code>
12	<code>val calculatedTip = amount.multiply(percentage)</code>

13	return if (isRoundUp) {
14	calculatedTip.setScale(0,
	RoundingMode.UP).setScale(2)
15	} else {
16	calculatedTip.setScale(2,
	RoundingMode.HALF_EVEN)
17	}
18	}
19	}

Tabel 5. Source Code MainActivity Compose Soal 1 Modul 2

1	package com.example.modul2
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.activity.enableEdgeToEdge
7	import androidx.compose.foundation.layout.Arrangement
8	import androidx.compose.foundation.layout.Column
9	import androidx.compose.foundation.layout.Row
10	import androidx.compose.foundation.layout.fillMaxSize
11	import androidx.compose.foundation.layout.fillMaxWidth
12	import androidx.compose.foundation.layout.padding
13	import
	androidx.compose.foundation.text.input.TextFieldLineLimits
14	import androidx.compose.material.icons.Icons
15	import androidx.compose.material.icons.filled.Money
16	import androidx.compose.material3.Icon
17	import androidx.compose.material3.Switch
18	import androidx.compose.material3.Text
19	import androidx.compose.material3.TextField
20	import androidx.compose.runtime.Composable
21	import androidx.compose.runtime.remember

```

22 import androidx.compose.ui.Alignment
23 import androidx.compose.ui.Modifier
24 import androidx.compose.ui.text.font.FontWeight
25 import androidx.compose.ui.tooling.preview.Preview
26 import androidx.compose.ui.unit.dp
27 import androidx.compose.ui.unit.sp
28 import com.example.modul2.ui.theme.Modul2Theme
29
30 class MainActivity : ComponentActivity() {
31     override fun onCreate(savedInstanceState: Bundle?) {
32         super.onCreate(savedInstanceState)
33         enableEdgeToEdge()
34         setContent {
35             Modul2Theme {
36                 TipCalculator()
37             }
38         }
39     }
40 }
41
42 @Composable
43 fun TipCalculator(
44     controller: TipCalculatorController = remember
45     { TipCalculatorController() }) {
46     Column(
47         modifier =
48         Modifier.fillMaxSize().padding(32.dp),
49         verticalArrangement =
50         Arrangement.spacedBy(16.dp, Alignment.CenterVertically),
51     ) {
52         Text(
53             text = "Calculate Tip",
54         )
55         TextField(

```

```

54         state = controller.billAmountState,
55         lineLimits = TextFieldLineLimits.SingleLine,
56         placeholder = { Text("Bill Amount") },
57         modifier = Modifier.fillMaxWidth(),
58         leadingIcon = {
59             Icon(
60                 imageVector = Icons.Default.Money,
61                 contentDescription = "Money Icon"
62             )
63         }
64     )
65
66     TipPercentageDropdown(
67         selected = controller.selectedPercentage,
68         onSelected = controller::updatePercentage,
69         modifier = Modifier.fillMaxWidth()
70     )
71
72     Row(
73         modifier = Modifier.fillMaxWidth(),
74         verticalAlignment = Alignment.CenterVertically,
75         horizontalArrangement = Arrangement.SpaceBetween
76     ) {
77         Text(
78             text = "Round up tip?"
79         )
80         Switch(
81             checked = controller.isRoundUp,
82             onCheckedChange = controller::toggleRoundUp
83         )
84     }
85

```

86	Text(
87	text = "Tip Amount : \$\$
	{controller.tipAmount}",
88	fontSize = 32.sp,
89	fontWeight = FontWeight.Bold
90	)
91	}
92	}
93	
94	@Preview(showBackground = true)
95	@Composable
96	fun TipCalculatorPreview() {
97	Modul2Theme {
98	TipCalculator()
99	}
100	}

Tabel 6. Source Code TipPercentageDropdown Soal 1 Modul 2

1	package com.example.modul2
2	
3	import androidx.compose.material.icons.Icons
4	import androidx.compose.material.icons.filled.Percent
5	import androidx.compose.material3.DropdownMenuItem
6	import
	androidx.compose.material3.ExperimentalMaterial3Api
7	import
	androidx.compose.material3.ExposedDropdownMenuAnchorType
8	import androidx.compose.material3.ExposedDropdownMenuBox
9	import
	androidx.compose.material3.ExposedDropdownMenuDefaults
10	import androidx.compose.material3.Icon
11	import androidx.compose.material3.Text
12	import androidx.compose.material3.TextField
13	import androidx.compose.runtime.Composable

```

14 import androidx.compose.runtime.getValue
15 import androidx.compose.runtime.mutableStateOf
16 import androidx.compose.runtime.remember
17 import androidx.compose.runtime.setValue
18 import androidx.compose.ui.Modifier
19
20 @OptIn(ExperimentalMaterial3Api::class)
21 @Composable
22 fun TipPercentageDropdown(
23     selected: String,
24     onSelected: (String) -> Unit,
25     modifier: Modifier = Modifier
26 ) {
27     val options = listOf("15", "18", "20")
28     var expanded by remember { mutableStateOf(false) }
29
30     ExposedDropdownMenuBox(
31         expanded = expanded,
32         onExpandedChange = { expanded = !expanded },
33         modifier = modifier
34     ) {
35         TextField(
36             value = "$selected%",
37             onValueChange = {},
38             readOnly = true,
39             trailingIcon = {
40                 ExposedDropdownMenuDefaults.TrailingIcon
41                 (expanded = expanded)
42             },
43             modifier = modifier
44                 .menuAnchor(ExposedDropdownMenuAnchorType.PrimaryNotEditable),
45             leadingIcon = {
46                 Icon(

```

46	imageVector = Icons.Default.Percent,
47	contentDescription = "Percent Icon"
48	)
49	}
50	)
51	
52	ExposedDropdownMenu(
53	expanded = expanded,
54	onDismissRequest = { expanded = false },
55	modifier = modifier
56	) {
57	options.forEach { option ->
58	DropdownMenuItem(
59	text = { Text("\$option%") },
60	onClick = {
61	onSelected(option)
62	expanded = false
63	},
64	contentPadding =
	ExposedDropdownMenuDefaults.ItemContentPadding
65	)
66	}
67	}
68	}
69	
70	}

Tabel 7. Source Code TipCalculatorController Soal 1 Modul 2

1	package com.example.modul2
2	
3	import
	androidx.compose.foundation.text.input.TextFieldState
4	import androidx.compose.runtime.derivedStateOf
5	import androidx.compose.runtime.getValue

6	import androidx.compose.runtime.mutableStateOf
7	import androidx.compose.runtime.setValue
8	import java.math.BigDecimal
9	
10	class TipCalculatorController(
11	val billAmountState: TextFieldState =
	TextFieldState()
12	) {
13	var selectedPercentage by mutableStateOf("15")
14	var isRoundUp by mutableStateOf(true)
15	
16	val tipAmount by derivedStateOf {
17	val amount =
	billAmountState.text.toString().toBigDecimalOrNull() ?:
	BigDecimal.ZERO
18	val percentage =
	selectedPercentage.toBigDecimal().divide(BigDecimal(100
	))
19	
20	TipCalculatorModel.calculateTip(amount,
	percentage, isRoundUp)
21	}
22	
23	fun updatePercentage(newPercent: String) {
24	selectedPercentage = newPercent
25	}
26	
27	fun toggleRoundUp(enabled: Boolean) {
28	isRoundUp = enabled
29	}
30	}

Tabel 8. Source Code TipCalculatorModel Soal 1 Modul 2

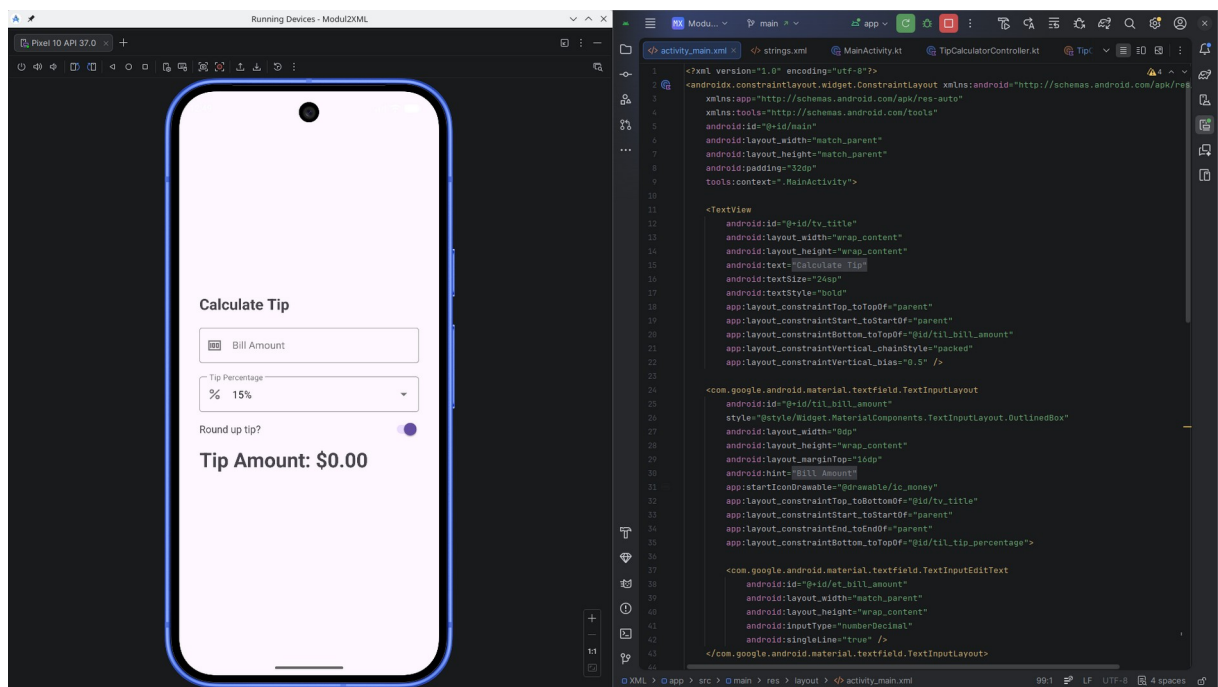
1	package com.example.modul2
2	

```

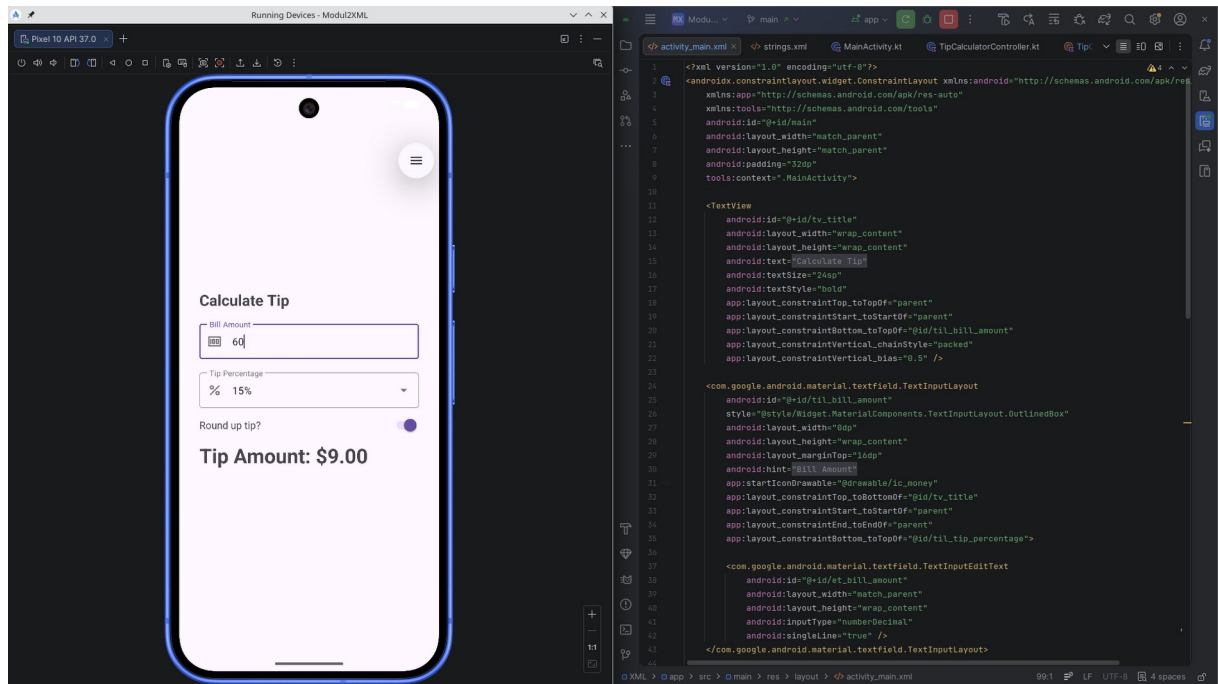
3 import java.math.BigDecimal
4 import java.math.RoundingMode
5
6 object TipCalculatorModel {
7     fun calculateTip(
8         amount: BigDecimal,
9         percentage: BigDecimal,
10        isRoundUp: Boolean
11    ): BigDecimal {
12        val calculatedTip = amount.multiply(percentage)
13        return if (isRoundUp) {
14            calculatedTip.setScale(0,
15RoundingMode.UP).setScale(2)
16        } else {
17            calculatedTip.setScale(2,
18RoundingMode.HALF_EVEN)
19        }
20    }
21 }

```

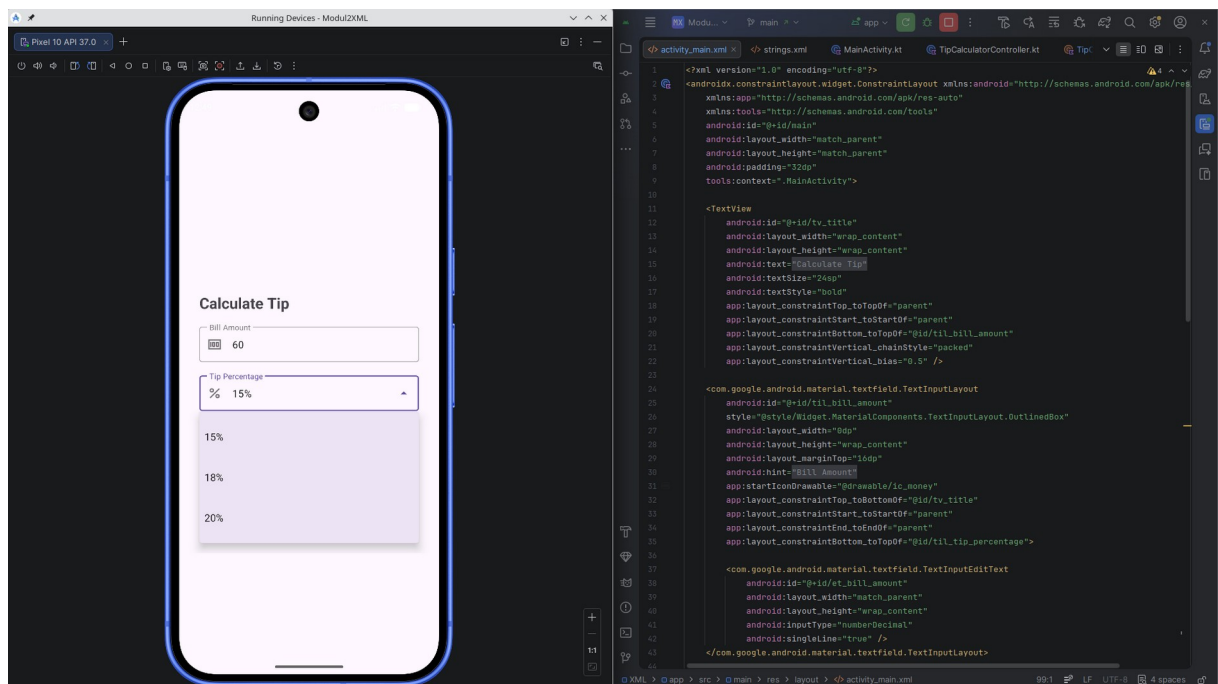
B. Output Program



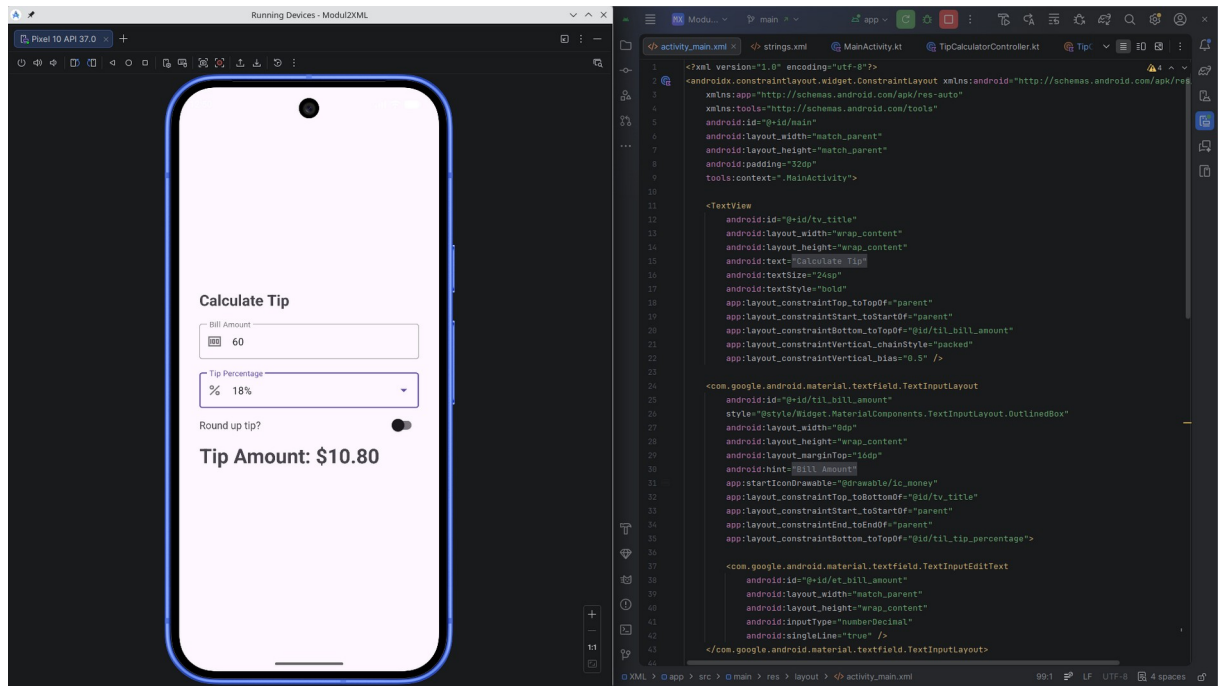
Gambar 4. Screenshot Output Soal 1 XML Modul 2 (1)



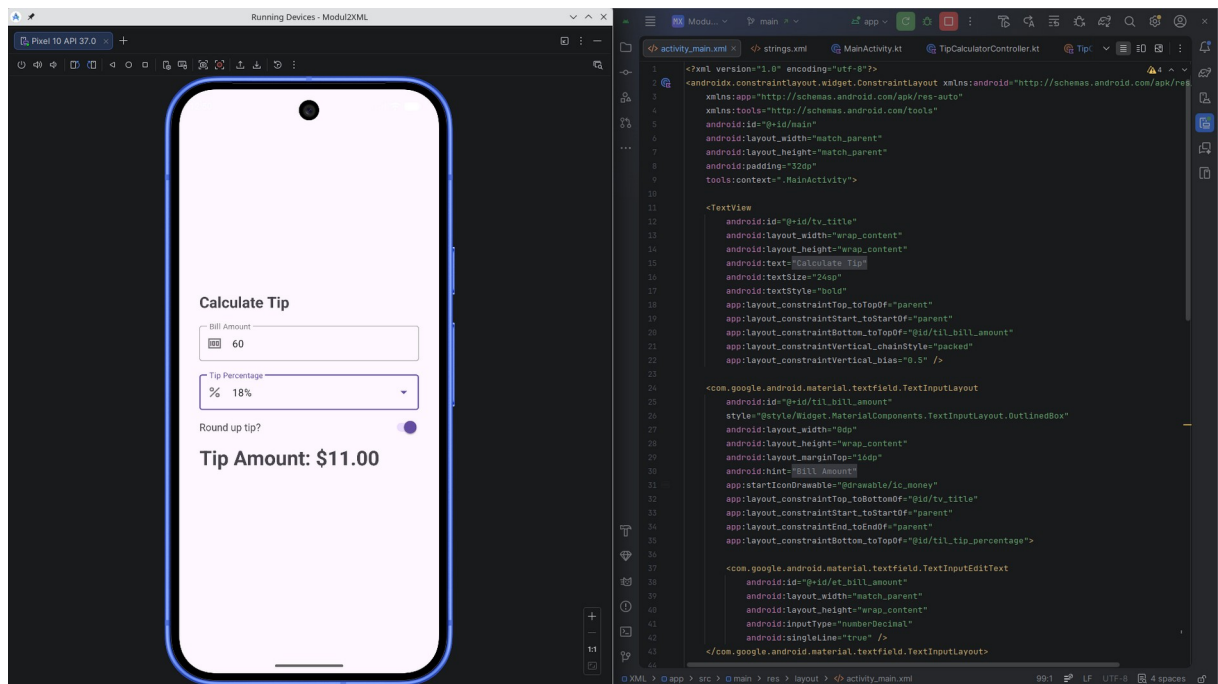
Gambar 5. Screenshot Output Soal 1 XML Modul 2 (2)



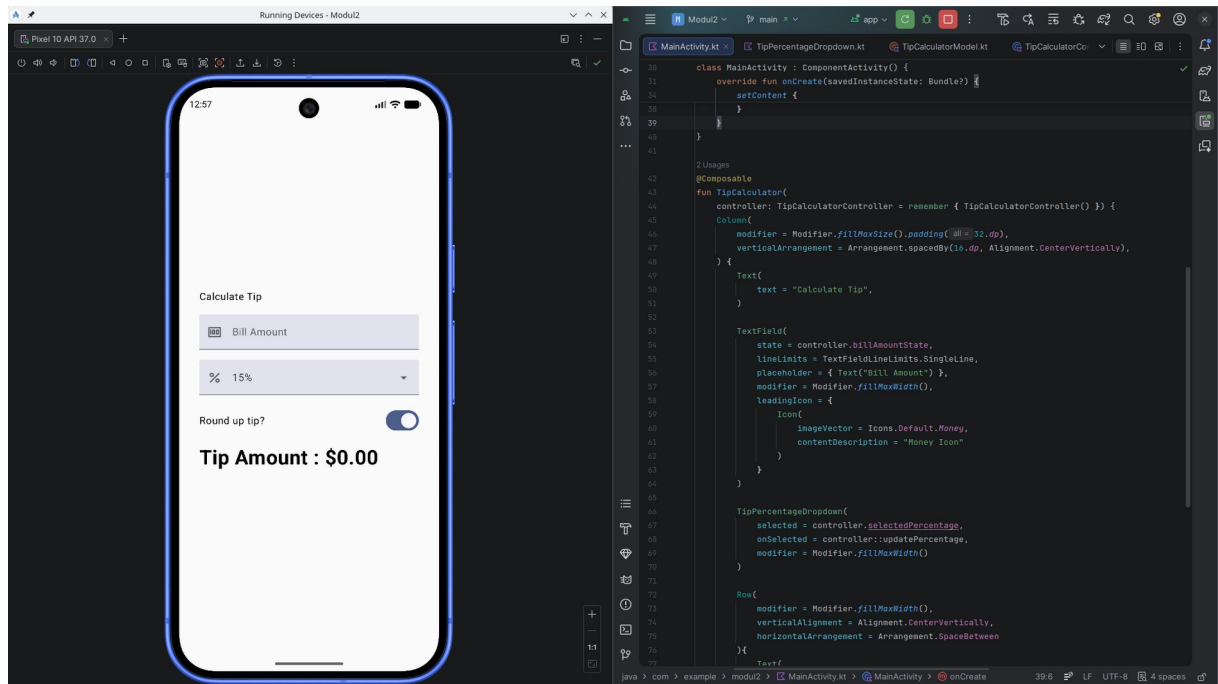
Gambar 6. Screenshot Output Soal 1 XML Modul 2 (3)



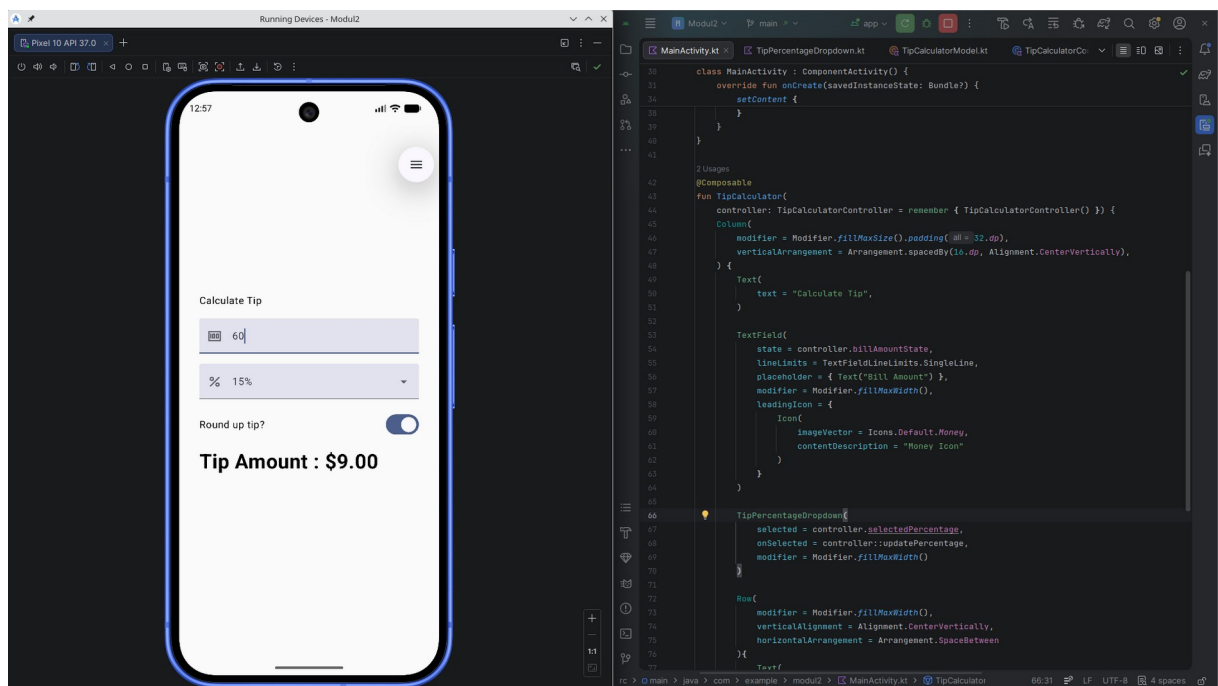
Gambar 7. Screenshot Output Soal 1 XML Modul 2 (4)



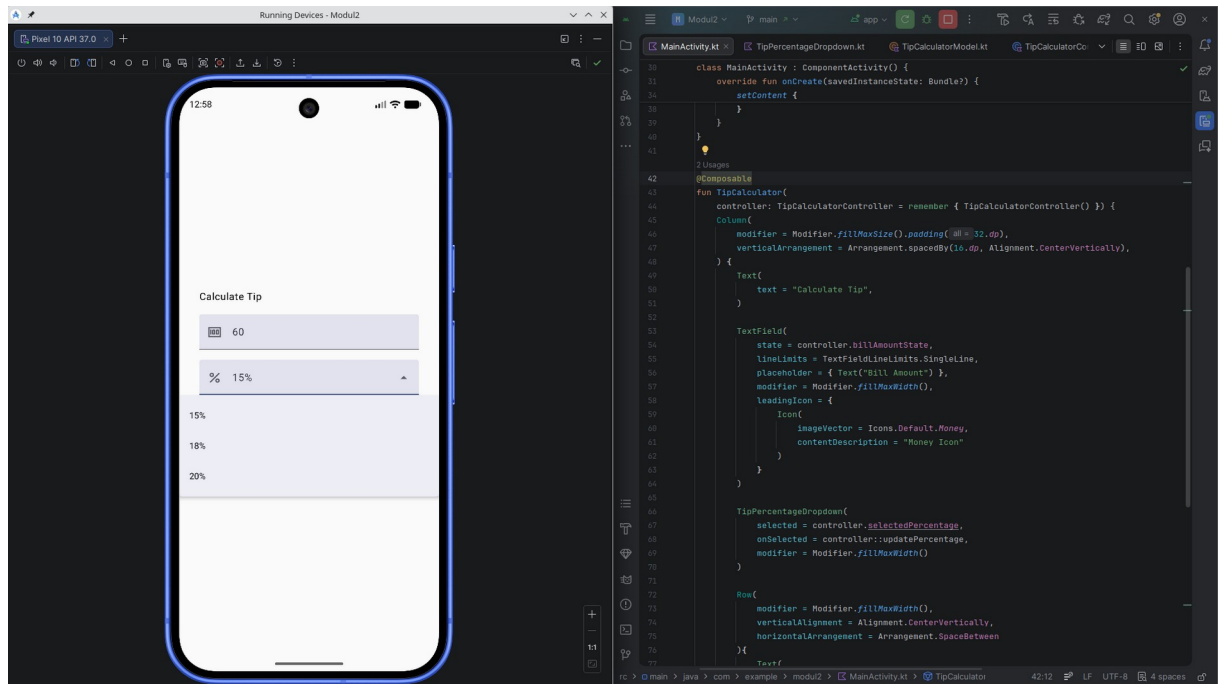
Gambar 8. Screenshot Output Soal 1 XML Modul 2 (5)



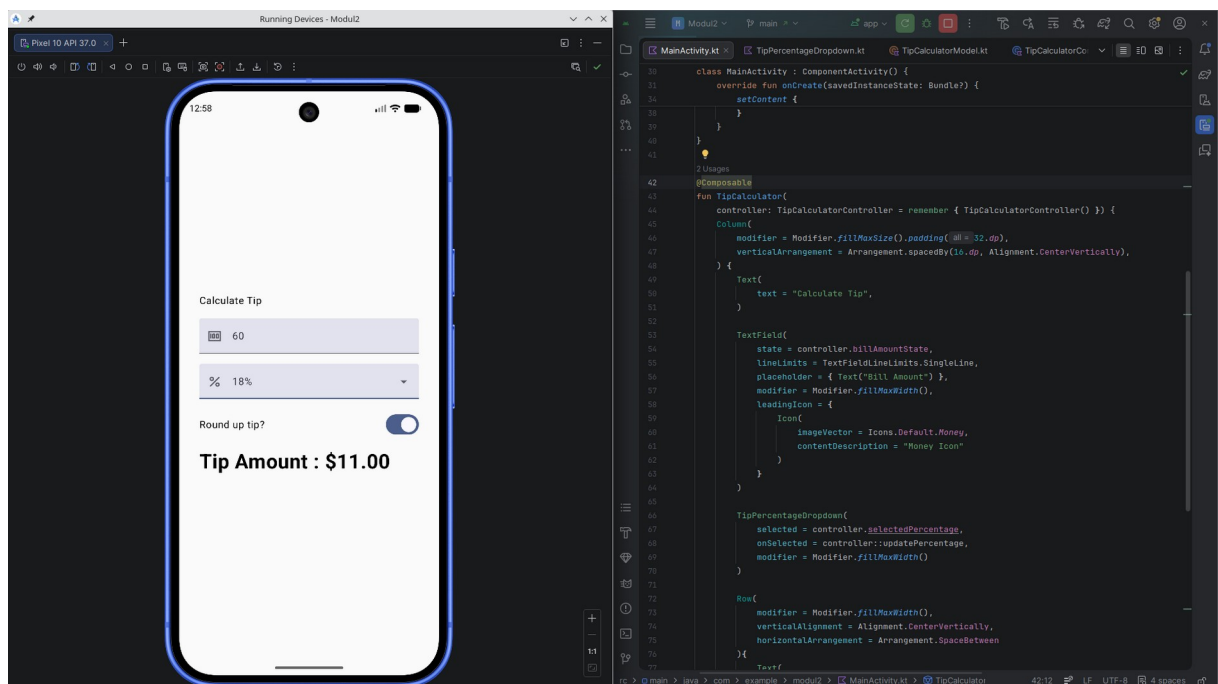
Gambar 9. Screenshot Output Soal 1 Compose Modul 2 (1)



Gambar 10. Screenshot Output Soal 1 Compose Modul 2 (2)



Gambar 11. Screenshot Output Soal 1 Compose Modul 2 (3)



Gambar 12. Screenshot Output Soal 1 Compose Modul 2 (4)

C. Pembahasan

Saya menggunakan MVC (Model View Controller) pada implementasi soal 1, hal ini untuk menghindari god class dan mempermudah maintenance jika saya ingin mengembangkan aplikasi ini lebih lanjut.

Perbedaan Implementasi XML dan Compose

Pada Implementasi XML, banyak sekali boilerplate seperti harus menggunakan view binding untuk melakukan referensi pada widget dan menggunakan listener untuk memonitor perubahan pada state seperti billAmount dan tipPercentage. Setiap kali ada perubahan input saya harus secara eksplisit memanggil method updateTipDisplay() untuk membuat UI sinkron dengan state di controller. Yang mana jika saya lakukan dalam jangka panjang rawan dengan error contohnya jika saya lupa menuliskan listener untuk fitur baru.

Pada Implementasi Compose, memiliki pendekatan data driven, jadi saya tidak perlu melakukan sinkronisasi manual antara state dan UI, state terhubung langsung dengan UI dengan menggunakan mutableStateOf.

Developer Experience saya saat mengimplementasikan XML sangat buruk karena saya harus bolak balik antara source code XML dan kotlin yang membuat saya sering kehilangan fokus karena context switching antara syntax XML dan kotlin yang sangatlah berbeda. Pada compose karena UI dan logika memiliki syntax yang sama yaitu kotlin saya memiliki beban pikiran yang lebih ringan daripada mengimplementasikan XML dan karena approach dari compose yang deklaratif saya bisa fokus pada apa yang ingin saya buat daripada bagaimana membuat hal tersebut.

Kelebihan dan Kelemahan Implementasi XML

Kelebihan

1. XML lebih solid daripada compose karena XML sudah lebih dulu daripada compose yang sekarang masih memiliki API experimental.
2. XML memberikan separation of concern antara UI dan logika dimana UI designer membuat UI menggunakan XML dan programmer membaut logika bisnis menggunakan kotlin

Kelemahan

1. XML sangatlah Verbosity, harus membuat boilerplate yang panjang untuk menghubungkan logika dengan layout
2. XML tidak memiliki single source of truth, state dapat berada di dua tempat yaitu di layout dan controller

Kelebihan dan Kelemahan Implementasi Compose

Kelebihan

1. Compose itu Deklaratif, jadi saya tidak dapat fokus membuat fitur ketika implementasi menggunakan Compose
2. Compose menerapkan single source of truth karena state dapat dipegang oleh controller saja
3. Maintainability pada Compose lebih baik daripada XML karena compose lebih mudah dibaca dan bisa langsung melakukan log.d atau debugging pada layout yang diperkirakan penyebab error

Kelemahan

1. Compose lebih susah untuk dipelajari oleh developer yang terbiasa dengan imperative
2. Compose berisiko membuat aplikasi melambat karena salah meletakkan state pada hierarki yang terlalu tinggi atau malah lupa menggunakan remember untuk state yang melakukan komputasi berat

D. Essay

1. Apa saja fitur yang menjadi daya jual pada Kotlin? (contoh: null safety, chain, dll)**
 Dari buku Kotlin in Action [1], dan artikel On the Adoption of Kotlin on Android Development: A Triangulation Study [2], saya mendapati 5 daya jual kotlin, yaitu:

1. Null Safety, Ini adalah fitur yang paling menarik. Di Java, NullPointerException adalah penyebab utama program crash. Kotlin mendesain tipe data secara default menolak null, sehingga potensi error ini sudah dicegah oleh compiler sejak kode pertama kali diketik, bukan saat aplikasi sudah berjalan di tangan pengguna [1].
2. Interoperability dengan Java, Kotlin dan Java bisa hidup berdampingan dalam satu proyek yang sama, bahkan saling menggunakan method satu sama lain tanpa error. Ini membuat proses migrasi aplikasi raksasa dari Java ke Kotlin bisa dilakukan secara bertahap tanpa harus menulis ulang semua source code dari nol [2].
3. Extension Functions, Fitur ini memungkinkan kita untuk menempelkan fungsi baru ke dalam sebuah Class yang sudah ada tanpa harus menggunakan Inheritance. Hal ini membuat kode menjadi jauh lebih ringkas dan mudah dibaca [1].
4. Coroutines, Jika Java menggunakan Thread yang sangat memakan memori untuk memproses tugas berat di latar belakang, Kotlin menggunakan Coroutines. Coroutines sangat ringan, memungkinkan developer menjalankan ratusan ribu proses secara bersamaan tanpa error Out of Memory [1].
5. Data Classes dan Sintaks Ringkas, Kotlin menghilangkan kewajiban menulis boilerplate yang panjang dengan memberikan banyak sekali magic. Pembuatan data

model yang mana di Java membutuhkan puluhan baris untuk getter, setter, dan equals, di Kotlin hanya membutuhkan satu baris kode saja [1].

2. berikan 1 test case terkait penggunaan fitur unggulan yang ada pada kotlin (contoh: penggunaan null safety)

Saya akan menggunakan salah satu fitur unggulan atau daya jual yang ada pada kotlin yaitu Interoperability dengan Java. Disini saya memiliki sebuah aplikasi lama. Data model user masih ditulis dalam Java. Saya akan membuat fitur notifikasi baru, tapi kali ini dalam bahasa kotlin agar lebih modern, tanpa menyentuh file Java yang sudah stabil.

Tabel 9. Source Code Java Data Model Modul 2

```
public class UserLegacy {  
    private String name;  
  
    public UserLegacy(String name) {  
        this.name = name;  
    }  
  
    public String getName() {  
        return name;  
    }  
}
```

Disini adalah data model yang ditulis menggunakan bahasa Java.

Tabel 10. Source Code Kotlin NotificationService Modul 2

```
class NotificationService {  
    fun sendWelcomeEmail(user: UserLegacy) {  
        println("Email terkirim: Halo ${user.name}, selamat  
datang di sistem baru kami!")  
    }  
}
```

Pada class Kotlin ini saya menggunakan data model yang diimplementasikan menggunakan bahasa Java tanpa adanya error.

Pada tabel di bawah saya menggunakan Java untuk menggunakan fitur notifikasi yang diimplementasikan menggunakan bahasa Kotlin. Fitur ini sangatlah mempermudah

developer dalam melakukan migrasi dari Java ke Kotlin tanpa mengorbankan stabilitas sistem.

Tabel 11. Source Code Java Main Modul 2

```
public class MainSystem {  
    public static void main(String[] args) {  
        UserLegacy user = new UserLegacy("Guntur");  
  
        NotificationService service = new  
NotificationService();  
        service.sendWelcomeEmail(user);  
    }  
}
```

DAFTAR PUSTAKA

- [1] S. Aigner, R. Elizarov, S. Isakova, and D. Jemerov, *Kotlin in action*, Second edition. Shelter Island: Manning, 2024.
- [2] V. Oliveira, L. Teixeira, and F. Ebert, “On the Adoption of Kotlin on Android Development: A Triangulation Study,” in *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, Feb. 2020, pp. 206–216. doi: 10.1109/SANER48275.2020.9054859.

LINK GITHUB

<https://github.com/Gunturadhtya/Praktikum-Pemrograman-Mobile>